Mini Linux Shell with Background Job Management

Operating Systems Hackathon Project

C-based command-line application

TEAM 4
2520030490 CHERUKUMUDI SRI DHRUTHI
2520030593 KACHANA MAHIMA
2520030185 RAVI ABHISHIKTHA

Problem Statement

Traditional command-line shells provide powerful process management capabilities, but understanding their internal working requires practical knowledge of operating-system concepts.

Our project aims to develop a lightweight Linux shell that demonstrates command execution, background job management, process monitoring, and process termination.

Key Challenges

- Executing system commands efficiently.
- Running multiple tasks without blocking the shell.
- Tracking and managing background processes.
- Handling process termination and signals safely.

Proposed Solution: A modular, C-based command-line shell that demonstrates fundamental Linux process-management mechanisms.

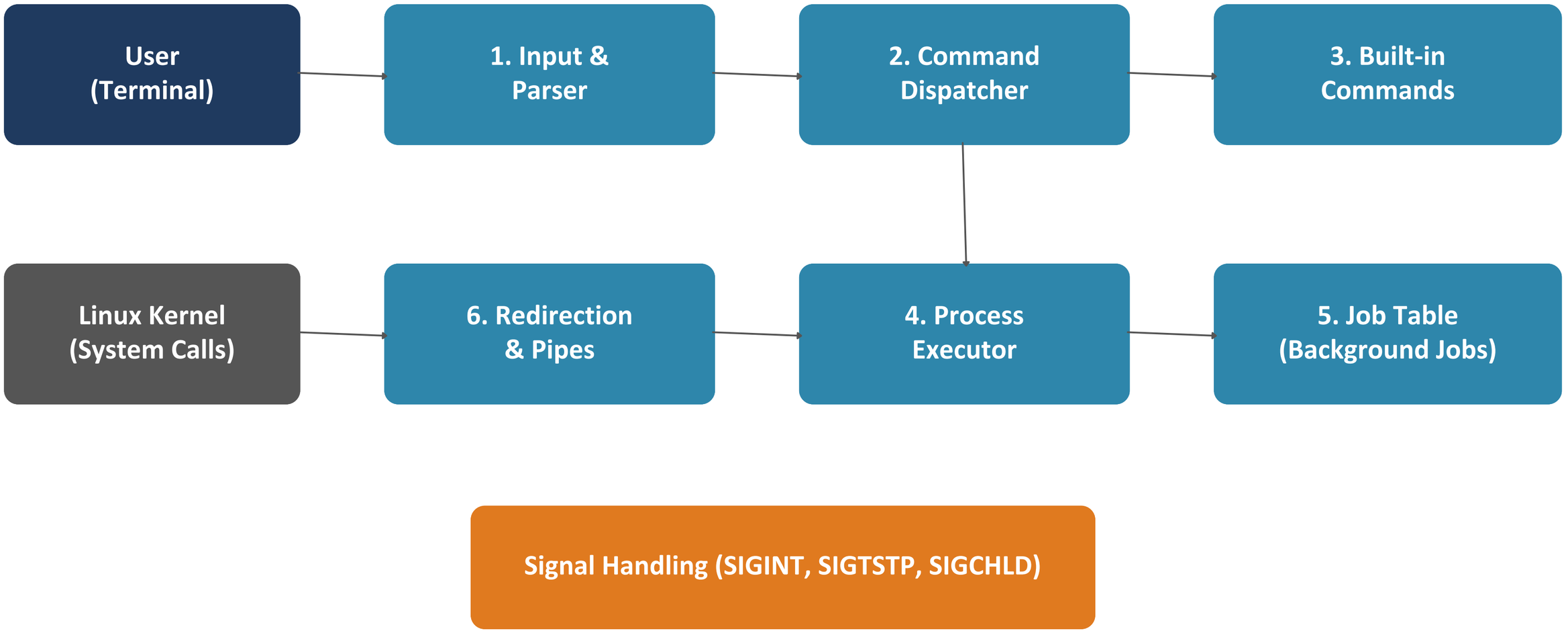
Objectives

Project Objectives

- Develop a functional command-line shell using C.
- Execute Linux commands using process-creation and execution system calls.
- Support foreground and background command execution.
- Maintain a job table to monitor background processes.
- Implement process control and signal handling.
- Demonstrate file I/O, pipes, and process synchronization where supported.
- Apply Operating Systems concepts through a practical implementation.

Expected Outcome: A working Linux shell that demonstrates process management through a simple terminal interface.

System Architecture



Dispatcher decides: built-in or external command, foreground or background.

System Workflow

User Input: Accept commands through the terminal.

Command Parser: Identify the command, arguments, and execution mode.

Command Dispatcher: Determine whether the input is a built-in or external command.

Command Executor: Create child processes and execute external programs.

Job Manager: Maintain background-job information and process states.

Signal Handler: Handle supported signals and child-process completion.

I**/O Manager:** Handle input/output redirection and pipes, if implemented.

Kernel Interaction: The shell uses Linux system calls to request process creation, program execution, synchronization, and file operations.

Six Modules

1. Input & Parser

Reads a line, splits it into command, arguments, & flag

2. Command Executor

Creates child with fork(), runs program with execvp()

3. Built-in Commands

cd, exit, jobs, fg, bg handled inside the shell

4. Background Job Manager

Stores jobs, shows status, reaps finished jobs

5. Signal Handler

Handles Ctrl+C, Ctrl+Z and child termination

6. Redirection & Pipes

Supports >, <, and | using dup2() and pipe()

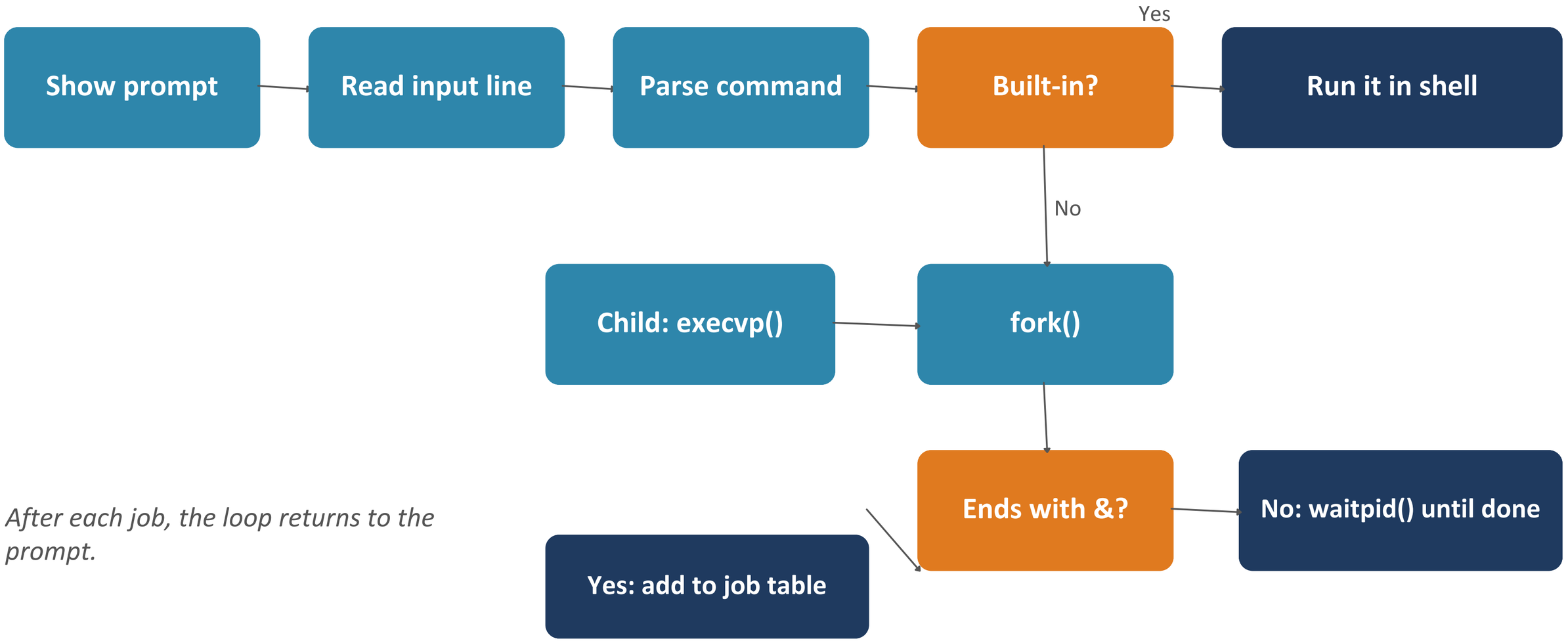
Modules: Key System Calls

Module	Main functions / system calls
Input & Parser	fgets(), strtok()
Command Executor	fork(), execvp(), waitpid()
Built-in Commands	chdir(), exit(), getcwd()
Background Job Manager	job table (struct array), waitpid(WNOHANG), kill()
Signal Handler	sigaction(), SIGINT, SIGTSTP, SIGCHLD
Redirection & Pipes	open(), dup2(), pipe(), close()

Technologies Used

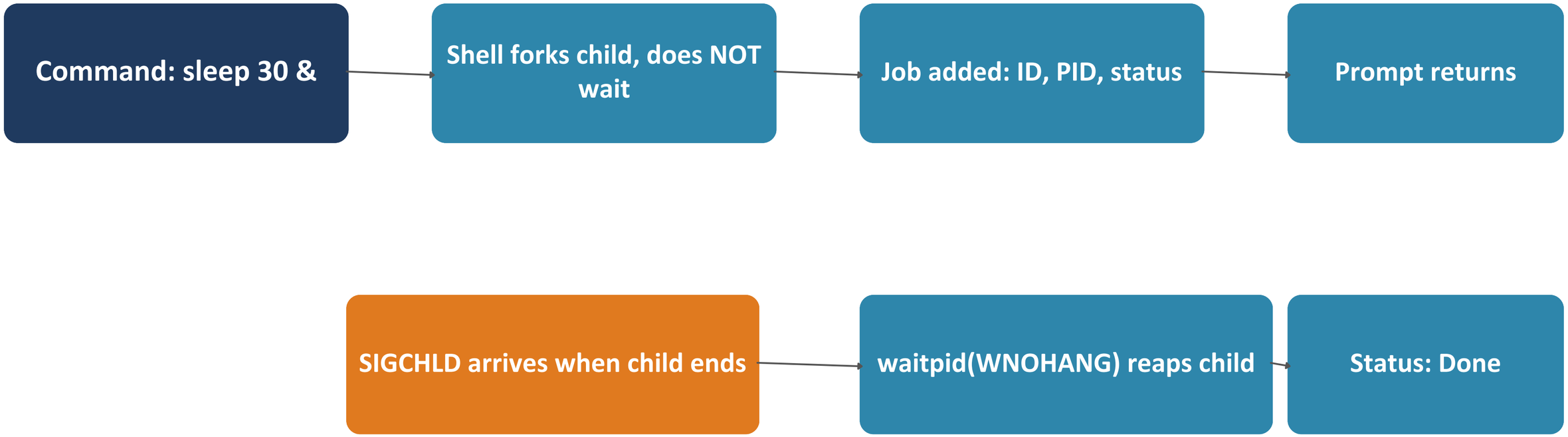
- Language: C (GCC compiler)
- Platform: Ubuntu / Linux
- APIs: POSIX system calls (unistd.h, sys/wait.h, signal.h, fcntl.h)
- Build tool: Makefile
- Tools: Git, text editor / VS Code terminal, GDB (debugging)
- Interface: command line only - no GUI

Implementation: Main Shell Loop



After each job, the loop returns to the prompt.

Background Job Management



jobs lists jobs | **fg %n** brings job to foreground | **bg %n** resumes it in background

Testing

Test case	Expected behaviour	Status
ls, pwd, echo	Output printed, prompt returns	[Pass
cd <dir>	Directory changes	[Pass
sleep 10 &	Prompt returns immediately	Pass
jobs	Shows running / done jobs	[Pass
fg / bg	Job moves foreground / background	[Pass
Ctrl+C	Stops foreground job, shell stays alive	[Pass
ls > out.txt, ls | wc	Redirection and pipe work	Pass
Invalid command	Error message, no crash	Pass

Fill the Status column from your real test runs.

Mapping to Operating Systems COs

CO	Topic (replace with your syllabus wording)	Where used in project
CO1	OS basics, system calls	fork, exec, wait, chdir
CO2	Process management	Foreground / background processes, job table
CO3	CPU scheduling & concurrency	Parallel background jobs, signals
CO4	Memory management	Buffers, dynamic memory, process images
CO5	Synchronization & IPC	Pipes, SIGCHLD handling
CO6	File systems & I/O	open, dup2, redirection

Check CO1-CO6 wording against your official course outcomes before submission.

Conclusion

- A mini shell in C that runs commands and manages background jobs on Linux.
- Shows how processes, signals, pipes and file I/O work together.
- All six modules cover the six OS course outcomes.
- Future work: command history, tab completion, job control improvements.
- Thank you - Questions?